

10.85inch e-Paper HAT+ (G) — driver does not work on Raspberry Pi OS Trixie

Product: 10.85inch e-Paper HAT+ (G), 1360×480, 4-colour **Status:** Resolved by changing OS. Reported so the driver or documentation can be updated.

Summary

Your driver for this panel — both the C and Python versions — hangs indefinitely on **Raspberry Pi OS Trixie (Debian 13)** and works correctly on **Bookworm (Debian 12)** on identical hardware.

The failure point is precise: `Init()` sends command `0x04` (`POWER_ON`), the panel asserts `BUSY`, and `BUSY` is never released. Every driver blocks there forever.

This affects your own demo code, unmodified, downloaded from your wiki. No customer code is involved.

Raspberry Pi OS Trixie is now the current release, so this will affect any customer starting a new installation with a fresh image.

Environment

	Fails	Works
OS	Raspberry Pi OS Trixie (Debian 13)	Raspberry Pi OS Bookworm (Debian 12)
Kernel	6.18.34+rpt-rpi-v6	6.6.x / 6.12.x
Board	Raspberry Pi 3 Model B Rev 1.2	Raspberry Pi 3 Model B Rev 1.2 (same unit)
Arch	32-bit Lite	32-bit Lite
Supply	5.1V regulated, <code>vcgencmd</code> <code>get_throttled = 0x0</code>	identical
Demo	10.85inch_e-Paper_G.zip from your wiki	same package

Same panel, same HAT, same Pi, same power supply, same demo package. Only the SD card image differs.

Symptom

Your C demo on Trixie:

```
Debug: EPD_10in85g_test Demo
set wiringPi lib success !!!
Debug: e-Paper Init and Clear...
Debug: e-Paper busy
Debug: e-Paper busy release
Debug: e-Paper busy
[hangs indefinitely]
```

The same binary and package on Bookworm completes all refresh passes and renders correctly.

Instrumented trace (Trixie)

A test program replicating `EPD_10in85g_Init()` exactly, sampling `BUSY` at each stage:

```
DEV_Module_Init() ok
    [after module init          ] BUSY=0
-- reset (20ms hi / 10ms lo / 20ms hi) --
    [RST high                   ] BUSY=1
    [RST low                     ] BUSY=0
    [RST high again              ] BUSY=1
    waiting on BUSY (post-reset)
    -> released after 0.00s
-- register configuration --
    [after 0x4D                  ] BUSY=1
    [after 0x00 (panel setting)  ] BUSY=1
    [after 0x06 x2 (booster)     ] BUSY=1
    [after 0x50                  ] BUSY=1
    [after 0x61 (resolution)     ] BUSY=1
    [after 0x65                  ] BUSY=1
    [after 0xE0/E3/E5/E9        ] BUSY=1
-- POWER_ON (0x04) --
    [immediately after 0x04      ] BUSY=1
    waiting on BUSY (power-on), starts at 0, timeout 45s
    -> TIMEOUT after 45s, BUSY still 0 (0 transitions)
```

Note the important detail: GPIO is not grossly broken on Trixie. `BUSY` tracks `RST` exactly — asserted while held in reset, released when reset lifts. The panel also asserts `BUSY` within milliseconds of `0x04`. So `RST` output and `BUSY` input both function, and the controller is receiving and acting on at least some commands.

What fails is only the completion of the power-up.

Eliminated on Trixie

Before the OS was identified as the cause, the following were ruled out:

- **Panels** — two units (original and RMA replacement)
- **Driver HATs** — two units (original and RMA replacement)
- **Raspberry Pi models** — Zero 2 W, Zero W, 3 Model B
- **GPIO backend** — the C demo built against **both** WiringPi and bcm2835; both hang identically
- **Driver implementation** — eight variants including your C demo, your Python demo, a ctypes wrapper around your C driver, and pure-Python drivers using kernel SPI
- **Kernel SPI interface** — `dtparam=spi=on` and disabled
- **Power supply** — four supplies including a 5.1V regulated unit with `throttled=0x0` throughout
- **Scheduler preemption** — running at `SCHED_FIFO` priority 99 via `chrt`
- **Reset timing** — six `RST` low durations from 2000 μ s to 50 μ s
- **Cabling** — factory interposer assembly and panel FPC direct to HAT
- **Command sequence variations** — 70 attempts across 7 variants (repeated `POWER_ON`, `POWER_OFF`/`POWER_ON` cycling, configuration sent twice, `CS_M` only, `CS_S` only, fixed delays instead of `BUSY` polling): 0/70

A clean-room install — fresh SD card, only `git`, `build-essential`, WiringPi and your demo package — reproduced the failure on Trixie.

Candidate mechanisms

Offered as hypotheses for your engineers; not verified here.

1. gpiomem device rename. The generic GPIO memory device has been renamed from `bcm2835-gpiomem` to `raspberrypi-gpiomem` in recent Raspberry Pi kernels, to reflect that it can now map RP1 ranges on Pi 5. Both `bcm2835` and `WiringPi` map GPIO registers directly rather than going through the kernel GPIO subsystem, so either could be affected by device naming or permission changes.

2. Peripheral base or register mapping. Both failing backends use direct register access. If base address detection or mapping behaviour changed in kernel 6.18, partial or subtly incorrect access could produce exactly this pattern — some pins working while transfers do not land correctly.

3. Bit-banged SPI timing. `DEV_SPI_WriteByte()` toggles SCLK and MOSI with no delays and no interrupt masking, so the SPI clock rate is whatever the CPU and library achieve. If register access latency changed between kernels, the resulting clock frequency or duty cycle may fall outside the controller’s tolerance. This would explain the configuration registers appearing to be accepted while `POWER_ON` fails — the panel may be receiving corrupted configuration data.

Hypothesis 3 fits the evidence best: RST and BUSY are single-pin operations and work correctly, while the SPI data path is timing-sensitive and is where the failure appears.

Note that Raspberry Pi Ltd now recommends `libgpiod` / `libgpiolib` over direct register access, and has committed to supporting it across all models. A driver backend built on that would be less exposed to this class of breakage.

Requested

1. Please test this panel’s demo package on Raspberry Pi OS Trixie (Debian 13).
2. If reproduced, either update the driver or add a documented OS requirement to the wiki. At present the wiki gives no OS version guidance, and Trixie is what a customer gets from Raspberry Pi Imager by default.
3. If you can identify the mechanism, that would be useful to publish — several of your panels share this `DEV_Config` layer, so other models are likely affected.

Environment details

- WiringPi 3.18 (maintained fork, github.com/WiringPi/WiringPi)
- `bcm2835` library 1.71
- `DEV_Config.h`: `Hardware_SPI 0`, `SPI_line 1` — SPI bit-banged in software over GPIO10 (MOSI) and GPIO11 (SCLK)
- Pin mapping: RST 17, DC 25, CS_M 8, CS_S 7, BUSY 24, PWR 18, MOSI 10, SCLK 11
- Demo package: `10.85inch_e-Paper_G.zip` from the product wiki